1. HashMap Internal Working

A HashMap is essentially an **Array of Buckets**. Each bucket is a linked list (or a tree) of Node objects.

The Step-by-Step Flow:

1. **Hashing:** When you call put(K, V), Java calls K.hashCode(). It then applies a supplemental hash function to prevent bad hash distributions.
2. **Indexing:** The hash is mapped to an index in the internal array using the formula: $index = hash \& (n-1)$ (where n is the array capacity).
3. **Storage:**
    - If the bucket is empty, the Node is placed there.
    - If a node already exists, it checks for a **Collision**.

---

2. Hash Collisions

A collision occurs when two different keys result in the same array index.

How Java Handles Collisions:

- **Chaining:** Initially, Java uses a **Singly Linked List**. New nodes are appended to the list at that index.
- **Treeification (Java 8+):** If a single bucket exceeds a threshold (8 nodes) and the total map capacity is ≥ 64, the linked list is converted into a **Red-Black Tree**.
    - **Why?** Searching a Linked List is $O(n)$. Searching a Red-Black Tree is $O(\log n)$. This prevents "Hash Denial of Service" attacks where someone intentionally sends keys that collide to slow down your system.

---

3. equals() vs hashCode()

These two methods form the "Contract" of Java collections.

- **The Contract:**
    1. If a.equals(b) is true, then a.hashCode() **must** be equal to b.hashCode().

2. If a.hashCode() == b.hashCode(), a.equals(b) **might** be true (but isn't guaranteed).

- **The Scenario:** Imagine a User object as a key.
    - If you override equals() to compare userId but forget hashCode(), two "equal" users will end up in different buckets because they have different default hash codes. You will never be able to retrieve the user you just put in the map!

---

4. ArrayList vs LinkedList

Both implement the List interface, but their memory layouts are polar opposites.

Feature	**ArrayList**	**LinkedList**
Structure	Dynamic Resizable Array	Doubly Linked List
Random Access	$O(1)$ (Direct index access)	$O(n)$ (Must traverse from head)
Insert/Delete	$O(n)$ (Due to shifting elements)	$O(1)$ (If you are already at the node)
Memory	Less (Contiguous block)	More (Every node stores 2 pointers)

Real-world Choice: In 99% of production cases, use ArrayList. Modern CPUs are optimized for contiguous memory (CPU Cache hits). LinkedList nodes are scattered in memory, leading to "Cache Misses."

---

5. Set vs List

- **List (The Sequence):** An ordered collection that allows duplicates. Use this when the **order of arrival** matters (e.g., a list of log events).
- **Set (The Collection):** An unordered collection that forbids duplicates. Use this when you need **uniqueness** (e.g., a set of unique User IDs).
    - *Implementation Detail:* A HashSet actually uses a HashMap internally. When you add a value to a HashSet, it puts it into a HashMap as a **Key** with a dummy constant as the **Value**.

---

6. ConcurrentHashMap vs HashMap

This is the most critical comparison for multi-threaded backend systems.

- **HashMap:** Not thread-safe. If two threads call put() simultaneously during a "rehash" (resizing), it can cause an infinite loop or data corruption.
- **ConcurrentHashMap (CHM):**
    - **Old Java (Pre-8):** Used "Segment Locking" (locking 16 different parts of the map).
    - **Modern Java (8+):** Uses **CAS (Compare-And-Swap)** operations for empty buckets and **synchronized blocks** only on the first node of a bucket during collisions.
- **Key Advantage:** CHM allows **multiple threads to read and write simultaneously** in different buckets without locking the entire map. Hashtable or Collections.synchronizedMap() lock the *entire* map, which kills performance.

Final Interview Scenario:

If you are building the **Batch Log Ingestor** we discussed earlier:

- You would use an **ArrayList** to store the batch because you want fast, sequential access.
- If you need to ensure you don't process the same Log ID twice across different threads, you would use a **ConcurrentHashMap** as a shared "Seen IDs" cache.